

Skill-21

Name: Simran Jangid

Rollno: 2520030166

Section: 8

Aim:

To connect modules, validate interfaces, test end-to-end execution, handle errors, improve reliability, and refactor components. Also, to detect runtime errors, generate error messages, log failures, handle invalid syntax, recover gracefully, and test error scenarios.

Objective:

- To connect different modules in a program.
- To validate interfaces between modules.
- To test complete end-to-end execution.
- To detect and handle runtime errors.
- To generate meaningful error messages.
- To log program failures.
- To detect invalid syntax or commands.
- To recover gracefully from errors.
- To improve program reliability.
- To refactor components for better maintainability.
- To test different error scenarios.

Problem Statement:

Design and implement a modular C program that connects multiple components and validates communication between them. The program should execute an end-to-end workflow, detect runtime and invalid-input errors, generate appropriate error messages, log failures, recover gracefully where possible, and demonstrate reliable and maintainable program execution.

Theory:

1. Modular Programming

Modular programming divides a program into smaller components or modules. Each module performs a specific task and can be developed and tested independently.

2. Module Connection

Different modules can be connected through functions and interfaces. One module can call functions provided by another module.

3. Interface Validation

An interface defines how modules communicate. Input values, return values, and error conditions should be checked to ensure that modules work correctly together.

4. End-to-End Execution

End-to-end testing checks the complete flow of a program from input to final output rather than testing only individual functions.

5. Runtime Errors

Runtime errors occur while a program is executing. Examples include invalid file operations, invalid input, and memory-related failures.

6. Error Messages

Error messages inform the user about what went wrong. Functions such as `perror()` can display system-related error information.

7. Error Logging

Errors can be recorded in a log file for later analysis. Logging helps identify failures and makes debugging easier.

8. Invalid Syntax

A command or input may contain invalid syntax. The program should detect the invalid input and display an appropriate message instead of terminating unexpectedly.

9. Graceful Recovery

Graceful recovery means handling an error without unnecessarily terminating the complete program. The program can report the error and continue with another operation when possible.

10. Refactoring

Refactoring improves the internal structure of a program without changing its intended functionality. Separating tasks into functions makes the program easier to understand and maintain.

Algorithm:

1. Start the program.
2. Define separate functions for input validation, command execution, error handling, and logging.
3. Connect the modules through function calls.
4. Read input from the user.
5. Validate the input.
6. Detect invalid syntax or commands.
7. Execute the requested operation.
8. Check for runtime errors.
9. Generate an appropriate error message if an error occurs.
10. Record the failure in a log file.
11. Recover from the error where possible.
12. Continue execution for valid operations.
13. Test the complete end-to-end workflow.
14. Display the final result.
15. Stop the program.

Data Used:

- User command/input.
- Command arguments.
- Return status of functions.
- Error messages.
- Log file.
- Process status.

Operations Used:

- `fgets()` for reading user input.
- `strcspn()` for removing the newline character.
- `strcmp()` for comparing commands.
- `system()` for executing commands.

- fopen() for opening the log file.
- fprintf() for recording errors.
- fclose() for closing the log file.
- Functions for modular program execution.

Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <errno.h>
```

```
void log_error(const char *msg) {
```

```
    FILE *f = fopen("error.log", "a");
```

```
    if (f == NULL) {
```

```
        perror("Log file error");
```

```
        return;
```

```
    }
```

```
    fprintf(f, "ERROR: %s\n", msg);
```

```
    fclose(f);
```

```
}
```

```
int validate_command(const char *cmd) {
```

```
    if (strcmp(cmd, "date") == 0)
```

```
        return 1;
```

```
    if (strcmp(cmd, "whoami") == 0)
```

```
        return 1;
```

```
    if (strcmp(cmd, "pwd") == 0)
        return 1;

    return 0;
}

int execute_command(const char *cmd) {
    int result = system(cmd);

    if (result == -1) {
        perror("Runtime error");
        log_error("Command execution failed");
        return 0;
    }

    return 1;
}

int main() {
    char command[100];

    printf("Skill - 21: Modular Error Handling\n");
    printf("-----\n");

    printf("Available commands: date, whoami, pwd\n");
    printf("Enter command: ");
```

```
if (fgets(command, sizeof(command), stdin) == NULL) {  
    perror("Input error");  
    log_error("Failed to read input");  
    return 1;  
}
```

```
command[strcspn(command, "\n")] = '\0';
```

```
if (strlen(command) == 0) {  
    printf("Error: Empty command.\n");  
    log_error("Empty command entered");  
    return 1;  
}
```

```
if (!validate_command(command)) {  
    printf("Error: Invalid command syntax.\n");  
    log_error("Invalid command entered");  
    return 1;  
}
```

```
printf("\nExecuting command...\n");
```

```
if (!execute_command(command)) {  
    printf("Error: Command execution failed.\n");  
    return 1;  
}
```

```
printf("Command executed successfully.\n");
```

```
    return 0;
}
```

Output:

```
simra@simran315:~$ nano skill21.c
simra@simran315:~$ gcc skill21.c -o skill21
simra@simran315:~$ ./skill21
Skill - 21: Modular Error Handling
-----
Available commands: date, whoami, pwd
Enter command: pwd

Executing command...
/home/simra
Command executed successfully.
simra@simran315:~$ ./skill21
Skill - 21: Modular Error Handling
-----
Available commands: date, whoami, pwd
Enter command: date

Executing command...
Fri Sep 18 04:57:57 UTC 2026
Command executed successfully.
simra@simran315:~$ ./skill21
Skill - 21: Modular Error Handling
-----
Available commands: date, whoami, pwd
Enter command: abc
Error: Invalid command syntax.
simra@simran315:~$ cat error.log
ERROR: Invalid command entered
```

Program Explanation:

16.1 Module Connection

The program is divided into separate functions:

- `log_error()` handles error logging.
- `validate_command()` validates the entered command.
- `execute_command()` executes the valid command.
- `main()` coordinates the complete workflow.

16.2 Interface Validation

The `validate_command()` function checks whether the entered command is one of the supported commands:

date

whoami

pwd

16.3 Error Detection

The program checks for several error conditions, including:

- Empty input.
- Invalid command.
- Input-reading failure.
- Command execution failure.
- Log-file opening failure.

16.4 Error Messages

When an invalid command is entered, the program displays:

Error: Invalid command syntax.

This gives the user a clear indication of the problem.

16.5 Error Logging

The `log_error()` function opens `error.log` and records the error:

ERROR: Invalid command entered

The log file is opened using append mode so previous error records are preserved.

16.6 Graceful Recovery

The program detects invalid input before attempting execution. Instead of executing an invalid command, it reports the error and terminates safely.

16.7 End-to-End Execution

The complete flow is:

Input

↓

Validation

↓

Module Connection

↓

Command Execution

↓

Error Checking

↓

Logging if Required

↓

Final Result

16.8 Refactoring

Separating validation, execution, and logging into individual functions improves code organization and makes each component easier to modify and test.

Testing:

- Tested valid command execution.
- Tested invalid command input.
- Tested empty input.
- Tested error-message generation.
- Tested error logging.
- Tested module-to-module function communication.
- Tested complete end-to-end execution.
- Verified that invalid commands were not executed.
- Verified that errors were recorded in the log file.
- Verified successful execution of supported commands.

Test Cases:

Test Case	Input / Action	Expected Result
1	date	Current date and time are displayed.
2	whoami	Current username is displayed.
3	pwd	Current working directory is displayed.
4	abc	Invalid command error is displayed and logged.

Test Case	Input / Action	Expected Result
5	Empty input	Empty-command error is displayed and logged.
6	Check error.log	Error messages are stored in the file.
7	Valid command after testing	Command executes successfully.

Observation:

- The program was divided into independent modules.
- The modules communicated successfully through function calls.
- Valid commands were identified correctly.
- Invalid commands were detected before execution.
- Appropriate error messages were generated.
- Errors were recorded in error.log.
- Runtime failures were handled using error checks.
- The complete end-to-end workflow executed successfully.
- The modular structure improved program organization and maintainability.

Result:

The modular error-handling program was successfully implemented. The program connected separate modules, validated user commands, executed valid operations, detected errors, generated error messages, logged failures, and handled invalid input safely.

Conclusion:

The practical demonstrated modular programming, interface validation, end-to-end execution, runtime-error detection, error logging, invalid-syntax handling, graceful error handling, and component refactoring in C. The program successfully performed both normal execution and error-scenario testing.